



Ejercicios de búsqueda con retroceso

Para cada ejercicio propuesto a continuación:

- Define una representación para las soluciones y las restricciones (explícitas e implícitas) que tienen que cumplir las tuplas solución.
- Describe el árbol del espacio de soluciones e indica su tamaño.
- Define los predicados acotadores que permiten podar ramas del árbol.

Ej. 1 — La distancia de Hamming entre dos cadenas de bits (enteros 0-1) es el número de posiciones de bit correspondientes que difieren. Se puede calcular usando el operador $\oplus$ (XOR). Por ejemplo:

$$\begin{array}{rcccccccccccc} A & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ B & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ \hline A \oplus B & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \end{array}$$

La distancia de Hamming en el ejemplo es 6, es decir es igual al número de 1 en la cadena $A \oplus B$. Diseña un algoritmo para imprimir todas las cadenas de bits de longitud n que tienen una distancia h de la cadena “nula”, es decir la cadena con todos los n bits a cero.

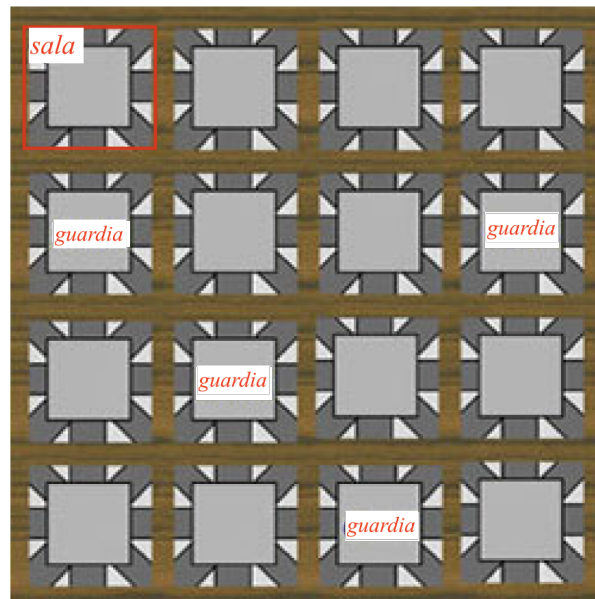
Ej. 2 — Diseña un algoritmo para decidir si puedes encontrar una expresión aritmética (donde las posibles operaciones son: suma y producto) que consiste en n números enteros no negativos dados $a_i (1 \leq i \leq n)$ y dará como resultado el valor E . La expresión aritmética será de la siguiente forma:

$$\left(\dots \left((a_{\pi(1)} \circ_1 a_{\pi(2)}) \circ_2 a_{\pi(3)} \right) \dots \right) \circ_{n-1} a_{\pi(n)}$$

donde $a_{\pi(1)} \dots a_{\pi(n)}$ es una permutación de los n números dados y $\circ_j \in \{+, *\} (1 \leq j \leq n-1)$ son las operaciones.

Ej. 3 — El señor Martin, director de una cadena de tiendas de fruta, a veces tiene ideas peculiares. Quiere introducir un nuevo producto: cestas de fruta. Las cestas contendrán todas las combinaciones posibles de fruta para satisfacer los clientes, pero como máximo una pieza de cada fruta. Hay n tipos de frutas, y quiere tener cestas de fruta con un peso entre 200 gramos y un kilo. Te pide calcular el número total de cestas diferentes que satisfacen esta condición y el peso total de todas las cestas. Para simplificar, se puede suponer que todas las frutas del mismo tipo pesan lo mismo y que toda las frutas pesan como mínimo 50 gramos. Diseña un algoritmo para resolver el problema.

Ej. 5 — El sultán de Brunei tiene un lugar secreto donde guarda su tesoro. Un ejemplo de mapa del sitio se puede ver en la figura, donde se considera un sitio 4×4 y hay 16 salas cuadradas (la sala de coordenadas $(1, 1)$ está resaltada en rojo).



Ej. 6 — Considera un grafo completo no dirigido con N vértices $\{1, \dots, N\}$ y A aristas, donde hay $k < A$ aristas que no se pueden recorrer (prohibidas). Diseña un algoritmo para calcular el número de ciclos hamiltonianos en el grafo que no incluyen las aristas prohibidas. Un grafo es completo si cada vértice está conectado a todos los demás vértices del grafo. Un ciclo hamiltoniano es un ciclo del grafo que visita cada vértice exactamente una vez. Los ciclos que incluyen las mismas aristas se cuentan solo una vez. Por ejemplo, los ciclos $\{1, 2, 3, 4, 1\}$, $\{1, 4, 3, 2, 1\}$ y $\{2, 3, 4, 1, 2\}$ son todos iguales (se cuentan una vez), mientras el ciclo $\{1, 3, 2, 4, 1\}$ es diferente de los anteriores.

Resolución: Cadenas de bits a distancia de Hamming h usando Backtracking

Este documento presenta la resolución detallada del problema de diseño de un algoritmo de Backtracking (Vuelta Atrás) para generar todas las cadenas de bits de longitud n que tienen exactamente una distancia de Hamming h respecto a la cadena nula (es decir, cadenas con exactamente h unos).

a) Representación de Soluciones y Restricciones

Para resolver este problema mediante backtracking, primero debemos definir de manera formal cómo se estructuran nuestras soluciones candidatas y qué reglas deben cumplir para ser válidas.

1. Representación de las Soluciones

Representamos cada solución como una tupla de tamaño n :

$$X = (x_1, x_2, \dots, x_n)$$

Donde cada componente x_i representa el valor del bit en la posición i .

2. Restricciones Explícitas

Las restricciones explícitas definen el dominio de cada variable de la tupla:

$$x_i \in \{0, 1\} \quad \forall i \in \{1, 2, \dots, n\}$$

Es decir, cada elemento de la secuencia solo puede tomar el valor de 0 o 1.

3. Restricciones Implícitas

Las restricciones implícitas determinan las condiciones que debe cumplir la tupla completa para ser considerada una solución válida del problema.

Dado que la distancia de Hamming respecto a la cadena nula $(0, 0, \dots, 0)$ es equivalente al número de bits establecidos en 1 (peso de Hamming), la restricción implícita exige que la suma de los componentes de la tupla sea exactamente h :

$$\sum_{i=1}^n x_i = h$$

b) Árbol del Espacio de Soluciones y su Tamaño

Descripción del Árbol

El espacio de búsqueda se representa mediante un **árbol de decisión binario**:

- **Raíz (Nivel 0)**: Representa el estado inicial antes de tomar ninguna decisión (ningún bit asignado).
- **Nivel i ($1 \leq i \leq n$)**: Representa la toma de decisión para la variable x_i .
- **Ramificación**: De cada nodo en el nivel $i - 1$ nacen dos ramas (hijos):
 - Una rama izquierda que asigna $x_i = 0$.
 - Una rama derecha que asigna $x_i = 1$.
- **Hojas (Nivel n)**: Representan asignaciones completas de la tupla $X = (x_1, \dots, x_n)$.

Tamaño del Árbol (Fuerza Bruta vs. Filtrado)

Si realizáramos una búsqueda por fuerza bruta sin ningún tipo de poda:

1. **Número de hojas (soluciones candidatas totales)**: 2^n
2. **Número total de nodos en el árbol**: $1 + 2 + 4 + \dots + 2^n = 2^{n+1} - 1$

Sin embargo, el número real de **soluciones válidas** (hojas que cumplen la restricción implícita) es mucho menor y está determinado por el coeficiente binomial:

$$\binom{n}{h} = \frac{n!}{h!(n-h)!}$$

El objetivo del Backtracking con predicados acotadores es evitar explorar aquellas ramas que sabemos con certeza que no nos conducirán a una de estas $\binom{n}{h}$ soluciones.

c) Predicados Acotadores (Pruning)

Para evitar la exploración de ramas infructuosas, introducimos criterios de poda en cada etapa de la recursión.

Supongamos que nos encontramos en el nivel k del árbol (es decir, ya hemos tomado decisiones para $x_1, x_2, \dots, x_{k-1}$ y nos disponemos a elegir el valor de x_k). Definimos la variable acumuladora u como la cantidad de unos que ya hemos colocado en la tupla:

$$u = \sum_{j=1}^{k-1} x_j$$

1. Poda por Exceso de Unos

Si en el paso actual ya hemos colocado más de h unos, es imposible que cualquier extensión de esta tupla sea válida (ya que no podemos "quitar" unos en los niveles siguientes).

- **Condición de poda**: $u > h$

2. Poda por Defecto de Unos

Si el número de unos que llevamos acumulados (u), sumado a la cantidad máxima de bits que nos quedan por decidir desde la posición actual hasta el final ($n - k + 1$), es menor que el total de unos requeridos (h), nunca podremos llegar a tener exactamente h unos aunque asignemos todos los bits restantes como 1.

- Condición de poda: $u + (n - k + 1) < h$

3. Predicado Acotador de Viabilidad

Por lo tanto, la rama actual es viable y debe seguir explorándose si y solo si se cumple la siguiente condición:

$$u \leq h \quad \wedge \quad u + (n - k + 1) \geq h$$

d) Algoritmo en Pseudocódigo

```

Procedimiento GenerarCadenasHamming(n, h)
Inicio
  // inicializar la tupla solución con ceros
  solucion_actual <- Arreglo de tamaño n inicializado en [0, 0, ..., 0]
  total_soluciones <- 0

  // sub-procedimiento recursivo interno
  Procedimiento Backtracking(nivel, unos_acumulados)
  Inicio
    // 1. PREDICADOS ACOTADORES (PODAS)
    // Poda por exceso de unos
    Si unos_acumulados > h Entonces
      Retornar
    FinSi

    // Poda por defecto de unos (posiciones restantes en el arreglo)
    posiciones_restantes <- n - nivel
    Si unos_acumulados + posiciones_restantes < h Entonces
      Retornar
    FinSi

    // 2. CASO BASE (Se ha completado la tupla de tamaño n)
    Si nivel = n Entonces
      Imprimir(solucion_actual)
      total_soluciones <- total_soluciones + 1
      Retornar
    FinSi

    // 3. RAMIFICACIÓN (Exploración de alternativas)

    // Opción A: Intentar asignando 0 en el bit de la posición actual
    solucion_actual[nivel] <- 0
    Backtracking(nivel + 1, unos_acumulados)

    // Opción B: Intentar asignando 1 en el bit de la posición actual
    solucion_actual[nivel] <- 1
    Backtracking(nivel + 1, unos_acumulados + 1)

  FinProcedimiento

```

```
// Llamada inicial desde el nivel 0 y con 0 unos acumulados
Backtracking(0, 0)
Imprimir("Cantidad total de soluciones válidas: ", total_soluciones)
FinProcedimiento
```

Análisis de la Complejidad con Podas

Gracias a las podas por exceso y por defecto, el algoritmo no explora las hojas inválidas. La complejidad temporal se reduce de un coste exponencial plano de $O(2^n)$ a un coste proporcional a la cantidad de soluciones reales generadas más las bifurcaciones internas viables, lo cual es asintóticamente óptimo:

$$O\left(\binom{n}{h}\right)$$

Resolución: Expresión Aritmética Objetivo mediante Backtracking

Este documento presenta la resolución detallada del problema de decidir si, dado un multiset de n números enteros no negativos $A = \{a_1, a_2, \dots, a_n\}$, existe una permutación de estos y una asignación de operadores de suma o producto evaluados de izquierda a derecha de la forma:

$$(((a_{\pi(1)} \circ_1 a_{\pi(2)}) \circ_2 a_{\pi(3)}) \cdots \circ_{n-1} a_{\pi(n)}) = E$$

a) Representación de Soluciones y Restricciones

Para modelar este problema mediante Backtracking, debemos representar de forma precisa tanto la permutación de los números como los operadores elegidos entre cada par.

1. Representación de las Soluciones

Una solución candidata se puede representar mediante una tupla mixta de tamaño $2n - 1$:

$$X = (p_1, o_1, p_2, o_2, \dots, o_{n-1}, p_n)$$

Donde:

- $P = (p_1, p_2, \dots, p_n)$ es una secuencia de índices que representa la permutación de los elementos del conjunto original A . Es decir, cada p_i indica el índice del elemento original a_{p_i} elegido en la posición i .
- $O = (o_1, o_2, \dots, o_{n-1})$ es la secuencia de operadores seleccionados para cada paso.

2. Restricciones Explícitas

Las restricciones explícitas definen los dominios de cada variable de decisión en la tupla:

- Para los índices de los operandos:

$$p_i \in \{1, 2, \dots, n\} \quad \forall i \in \{1, \dots, n\}$$

- Para los operadores aritméticos:

$$o_j \in \{+, *\} \quad \forall j \in \{1, \dots, n-1\}$$

3. Restricciones Implícitas

Las restricciones implícitas son las condiciones de validez que debe cumplir la tupla completa:

1. **Restricción de Permutación (Inyectividad):** No se puede reutilizar el mismo elemento de la entrada en posiciones diferentes de la expresión:

$$p_i \neq p_j \quad \forall i \neq j$$

2. **Restricción de Resultado Objetivo:** Si definimos el valor acumulado parcial v_k en el paso k como:

- $v_1 = a_{p_1}$
- $v_k = v_{k-1} \circ_{k-1} a_{p_k} \quad \forall k \in \{2, \dots, n\}$

La tupla solución debe cumplir estrictamente que el resultado final sea igual al valor objetivo E :

$$v_n = E$$

b) Árbol del Espacio de Soluciones y su Tamaño

Descripción del Árbol

El espacio de búsqueda se puede conceptualizar como un árbol donde alternamos la selección de un número disponible y la selección de un operador:

- **Raíz (Nivel 0):** Estado inicial. No se ha seleccionado ningún número ni ningún operador.
- **Nivel 1:** Selección del primer operando a_{p_1} . Hay exactamente n ramificaciones posibles (cualquiera de los elementos de A).
- **Nivel $2k$ (para $1 \leq k < n$):** Selección del operador o_k . Hay exactamente 2 ramificaciones posibles ($+$ o $*$).
- **Nivel $2k + 1$ (para $1 \leq k < n$):** Selección del $(k + 1)$ -ésimo operando $a_{p_{k+1}}$. Se ramifica en los $(n - k)$ elementos que todavía no han sido seleccionados.
- **Hojas (Nivel $2n - 1$):** Representan una expresión aritmética completa y totalmente evaluada.

Tamaño del Árbol (Búsqueda Exhaustiva)

Si realizamos una exploración de fuerza bruta sin ningún tipo de poda, el número de hojas (cadenas de decisiones completas evaluadas) se calcula de la siguiente manera:

1. Hay exactamente $n!$ maneras de ordenar (permutar) los n números de entrada.
2. Para cada una de estas permutaciones, hay $n - 1$ posiciones intermedias donde podemos elegir de forma independiente entre 2 operadores, lo que da 2^{n-1} combinaciones de signos.

Por tanto, el número total de hojas en el árbol es:

$$\text{Número de hojas} = n! \cdot 2^{n-1}$$

c) Predicados Acotadores (Pruning)

Dado que los números del multiset A son **enteros no negativos** ($a_i \geq 0$), podemos deducir reglas matemáticas sumamente potentes para descartar ramas del árbol que jamás alcanzarán el valor E .

Supongamos que estamos en el paso de decisión k , donde ya se ha acumulado un valor parcial v_k , y nos queda un conjunto R de números de la entrada sin utilizar (donde $|R| = n - k$).

1. Poda por Superación Inevitable (Exceso)

En aritmética de enteros no negativos, sumar o multiplicar por números mayores o iguales a 1 nunca disminuye el valor acumulado. El único número que puede disminuir nuestro acumulado es el 0 (al multiplicar por él, reduciendo el acumulado a 0).

Por tanto, si el valor parcial acumulado ya ha superado el objetivo E y ya **no nos quedan ceros** por colocar en nuestro conjunto de números restantes R , es imposible volver a bajar el valor para que sea igual a E .

- Condición de Poda: $(v_k > E) \quad \wedge \quad (0 \notin R)$

2. Poda por Cota Superior Insuficiente (Defecto)

Podemos estimar de manera óptima el valor máximo teórico que se podría llegar a alcanzar a partir de nuestro valor actual v_k utilizando los números restantes en R .

Para maximizar un valor con enteros no negativos, la mejor estrategia teórica consiste en multiplicar todos los números mayores que 1 y sumar los números restantes (como el 0 o el 1, con los que multiplicar no incrementaría el valor).

Definimos una cota superior segura $V_{\max}$ como:

$$V_{\max} = \left(v_k + \sum_{r \in R, r \leq 1} r \right) \times \prod_{r \in R, r > 1} r$$

(Nota: Si no hay elementos mayores a 1 en R , el producto se toma como 1).

Si esta cota máxima es estrictamente menor que el valor objetivo E , entonces no existe ninguna combinación de sumas y multiplicaciones con los elementos restantes que pueda alcanzar E .

- Condición de Poda:

$$V_{\max} < E$$

3. Predicado Acotador de Viabilidad

Un nodo en la profundidad k con un valor acumulado v_k y un conjunto de recursos restantes R es **viable** si y solo si pasa ambas pruebas:

$$\neg \left((v_k > E) \wedge (0 \notin R) \right) \quad \wedge \quad \left(V_{\max} \geq E \right)$$

d) Análisis de Coste (Complejidad Temporal y Espacial)

- **Complejidad Temporal:** $O(n! \cdot 2^{n-1})$ en el peor caso sin podas (por la generación de permutaciones y combinaciones de operadores), reduciéndose drásticamente en la práctica gracias a los predicados acotadores.
- **Complejidad Espacial:** $O(n)$ debido a la profundidad máxima de la pila recursiva ($2n - 1$ niveles en el árbol) y al uso del vector auxiliar de marcas.

Resolución: Cestas de Fruta del Señor Martín mediante Backtracking

Este documento presenta la resolución detallada del problema de diseño de un algoritmo de **Backtracking** (Vuelta Atrás) para calcular el número total de cestas de frutas posibles cuyo peso total se encuentre entre los 200 gramos y 1 de kilogramo (1000 gramos), además de obtener el peso acumulado de todas las cestas válidas combinadas.

a) Representación de Soluciones y Restricciones

1. Representación de las Soluciones

Representamos cada cesta como una tupla binaria de tamaño n :

$$X = (x_1, x_2, \dots, x_n)$$

Donde cada componente x_i representa la presencia o ausencia de la fruta de tipo i en la cesta:

- $x_i = 1$: Se incluye la fruta de tipo i en la cesta.
- $x_i = 0$: No se incluye la fruta de tipo i en la cesta.

Asumimos que disponemos de un vector constante de pesos $W = [w_1, w_2, \dots, w_n]$ que indica el peso de una pieza de cada tipo de fruta i (donde $w_i \geq 50$ gramos $\forall i$).

2. Restricciones Explícitas

Las restricciones explícitas definen el dominio binario de cada variable de decisión:

$$x_i \in \{0, 1\} \quad \forall i \in \{1, 2, \dots, n\}$$

3. Restricciones Implícitas

La restricción implícita exige que el peso total de las frutas seleccionadas en la cesta esté estrictamente dentro del rango $[200, 1000]$ gramos:

$$200 \leq \sum_{i=1}^n (w_i \cdot x_i) \leq 1000$$

b) Árbol del Espacio de Soluciones y su Tamaño

Descripción del Árbol

El espacio de búsqueda se representa mediante un **árbol de decisión binario**:

- **Raíz (Nivel 0)**: Cesta vacía sin evaluar ninguna fruta.

- **Nivel i ($1 \leq i \leq n$):** Representa la toma de decisión para la fruta de tipo i .
- **Ramificación:** De cada nodo en el nivel $i - 1$ se abren dos opciones:
 - Rama izquierda: Asigna $x_i = 0$ (no incluir la fruta).
 - Rama derecha: Asigna $x_i = 1$ (incluir la fruta).
- **Hojas (Nivel n):** Cestas con una composición completa evaluada para las n frutas.

Tamaño del Árbol (Búsqueda Exhaustiva)

Sin la aplicación de podas en el árbol de búsqueda:

1. Número de hojas (soluciones candidatas evaluadas): 2^n
2. Número total de nodos en el árbol:

$$1 + 2 + 4 + \dots + 2^n = 2^{n+1} - 1$$

c) Predicados Acotadores (Pruning)

Definimos P_{act} como el peso acumulado de las frutas que ya hemos decidido incluir hasta el nivel actual k :

$$P_{\text{act}} = \sum_{j=1}^{k-1} (w_j \cdot x_j)$$

Y definimos P_{rest} como la suma de los pesos de todas las frutas restantes sobre las que aún no hemos tomado una decisión (desde la fruta k hasta la n):

$$P_{\text{rest}} = \sum_{j=k}^n w_j$$

1. Poda por Exceso de Peso

Dado que los pesos de todas las frutas son estrictamente positivos ($w_i \geq 50$), añadir más elementos a la cesta nunca reducirá su peso. Si el peso actual de la cesta supera el límite de 1000 gramos, podemos abortar la exploración de esa rama de inmediato.

- Condición de poda:

$$P_{\text{act}} > 1000$$

2. Poda por Defecto de Peso

Si el peso que ya llevamos acumulado (P_{act}) sumado al peso de todas las frutas disponibles que quedan por decidir (P_{rest}) es menor que el mínimo requerido de 200 gramos, es imposible alcanzar el rango de validez aunque incluyemos todas las frutas restantes.

- Condición de Poda:

$$P_{\text{act}} + P_{\text{rest}} < 200$$

3. Predicado Acotador de Viabilidad

Un nodo en el nivel k es viable para continuar su exploración recursiva si y solo si cumple:

$$P_{\text{act}} \leq 1000 \quad \wedge \quad P_{\text{act}} + P_{\text{rest}} \geq 200$$

d) Algoritmo en Pseudocódigo y Análisis de Coste

A continuación se presenta el pseudocódigo recursivo que resuelve el problema contando las cestas válidas y acumulando su peso total.

```

Procedimiento ResolverCestasDeFruta(n, W)
Inicio
    total_cestas <- 0
    peso_total_acumulado <- 0

    // Calcular el peso de todas las frutas para la poda por defecto inicial
    suma_restante_inicial <- 0
    Para i <- 1 Hasta n Con Paso 1 Hacer
        suma_restante_inicial <- suma_restante_inicial + W[i]
    FinPara

    // Procedimiento recursivo de backtracking
    Procedimiento Backtracking(nivel, peso_actual, peso_restante)
    Inicio
        // 1. PREDICADOS ACOTADORES (PODAS)
        Si peso_actual > 1000 Entonces
            Retornar
        FinSi

        Si peso_actual + peso_restante < 200 Entonces
            Retornar
        FinSi

        // 2. CASO BASE (Se ha tomado una decisión para todas las frutas)
        Si nivel = n + 1 Entonces
            // El peso_actual está garantizado en el rango [200, 1000] por las pc
            total_cestas <- total_cestas + 1
            peso_total_acumulado <- peso_total_acumulado + peso_actual
            Retornar
        FinSi

        // 3. RAMIFICACIÓN
        // Opción A: No incluir la fruta del nivel actual (x_nivel = 0)
        // El peso acumulado no sube, pero el peso restante disponible disminuye
        Backtracking(nivel + 1, peso_actual, peso_restante - W[nivel])

        // Opción B: Incluir la fruta del nivel actual (x_nivel = 1)
        // El peso acumulado sube y el peso restante disponible disminuye
        Backtracking(nivel + 1, peso_actual + W[nivel], peso_restante - W[nivel])
    Fin
End

```

```
FinProcedimiento

// Iniciar el recorrido en el nivel 1 con peso actual de 0
Backtracking(1, 0, suma_restante_inicial)

Imprimir("Número total de cestas diferentes: ", total_cestas)
Imprimir("Peso total de todas las cestas: ", peso_total_acumulado)
FinProcedimiento
```

Análisis de Coste

- **Complejidad Temporal:** $O(2^n)$ en el peor caso teórico sin podas (árbol binario completo), reduciéndose notablemente en la práctica gracias a las cotas de peso.
- **Complejidad Espacial:** $O(n)$ debido a la profundidad máxima de la pila de llamadas de la recursión, la cual está acotada por el número total de frutas n .

Resolución: Coloreo Mínimo de Grafos mediante Backtracking

Este documento presenta la resolución detallada para encontrar el número cromático $\chi(G)$ de un grafo no dirigido $G = (V, E)$ usando **Backtracking** (Vuelta Atrás). El objetivo es asignar a cada vértice un color de tal manera que ningún par de vértices conectados compartan el mismo color, minimizando la cantidad total de colores empleados.

a) Representación de Soluciones y Restricciones

1. Representación de las Soluciones

Asumimos que el grafo tiene n vértices indexados de 1 a n , es decir, $V = \{v_1, v_2, \dots, v_n\}$, y un conjunto de aristas E . Representamos la coloración como una tupla de tamaño n :

$$X = (x_1, x_2, \dots, x_n)$$

Donde cada componente x_i representa el identificador numérico del color asignado al vértice v_i .

2. Restricciones Explícitas

En el peor de los casos (un grafo completo K_n), necesitaremos a lo sumo n colores diferentes. Por lo tanto, el dominio de cada variable es:

$$x_i \in \{1, 2, \dots, n\} \quad \forall i \in \{1, \dots, n\}$$

3. Restricciones Implícitas

Las restricciones implícitas que garantizan una solución válida y óptima son:

1. **Adyacencia (Restricción de Coloreo Válido):** Dos vértices conectados por una arista no pueden compartir el mismo color.

$$\forall (v_i, v_j) \in E \implies x_i \neq x_j$$

2. **Minimización (Restricción de Optimalidad):** Si definimos la cantidad de colores únicos usados en la tupla X como $C(X) = |\{x_1, \dots, x_n\}|$, buscamos minimizar este valor respecto a cualquier otra asignación válida:

$$\chi(G) = \min C(X)$$

Durante la ejecución, mantendremos una variable global $mejor_G$ que almacene la menor cantidad de colores con la que se ha logrado colorear el grafo completo hasta el momento.

b) Árbol del Espacio de Soluciones y su Tamaño

Descripción del Árbol

El espacio de búsqueda se estructura mediante un árbol de decisión n -ario:

- **Raíz (Nivel 0):** Grafo sin colorear. Ninguna decisión tomada.
- **Nivel i ($1 \leq i \leq n$):** Representa la asignación de color al vértice v_i .
- **Ramificación:** Desde un nodo en el nivel $i - 1$, se abren tantas ramas como colores disponibles decidamos probar para el vértice v_i . En un análisis de fuerza bruta sin restricciones, se prueban hasta n colores posibles.
- **Hojas (Nivel n):** Representan asignaciones completas de color para todos los vértices del grafo.

Tamaño del Árbol (Búsqueda Exhaustiva)

Si realizáramos una búsqueda por fuerza bruta evaluando todas las posibilidades de coloreo con hasta n colores:

1. Número de hojas (soluciones candidatas totales): n^n
2. Número total de nodos en el árbol:

$$\sum_{i=0}^n n^i = \frac{n^{n+1} - 1}{n - 1} \approx O(n^n)$$

c) Predicados Acotadores (Pruning)

Para resolver este problema de manera eficiente, el algoritmo de Backtracking debe incorporar tres estrategias de poda críticas para evitar la exploración del árbol completo:

1. Poda por Consistencia Local (Viabilidad Inmediata)

Al evaluar el color c para el vértice v_k , este no puede ser igual al color ya asignado a ninguno de sus vecinos adyacentes que ya hayan sido coloreados en los niveles anteriores.

- **Condición de Poda (Inviabilidad):**

$$\exists j < k \text{ tal que } (v_j, v_k) \in E \quad \wedge \quad x_j = c$$

2. Poda por Cota Superior (Optimización)

Si en el nivel actual k de la recursión ya hemos utilizado una cantidad de colores distintos que es igual o mayor que el mejor mínimo global de colores encontrado previamente ($mejor_C$), podemos podar la rama inmediatamente. Es imposible que esta rama produzca una solución con menos colores.

Sean $U_k = \{x_1, \dots, x_{k-1}\}$ el conjunto de colores ya utilizados en los niveles anteriores.

- **Condición de Poda:**

$$|U_k| \geq \text{mejor_C}$$

d) Algoritmo en Pseudocódigo y Análisis de Coste

A continuación se detalla el diseño del algoritmo recursivo en pseudocódigo para calcular el número mínimo de colores necesarios.

```

Procedimiento CalcularNumeroCromatico(n, E)
Inicio
  // Vector de asignación de colores inicializado en 0 (sin colorear)
  color_asignado <- Arreglo de tamaño n + 1 inicializado en [0, ..., 0]

  // El peor caso inicial para colorear un grafo de n vértices son n colores
  mejor_C <- n

  // Función auxiliar para verificar si un color es válido para el vértice actu
  Funcion EsSeguro(vertice, color_a_probar)
  Inicio
    Para vecino <- 1 Hasta n Con Paso 1 Hacer
      Si (vecino, vertice) en E y color_asignado[vecino] = color_a_probar E
        Retornar Falso
      FinSi
    FinPara
    Retornar Verdadero
  FinFuncion

  // Procedimiento recursivo de backtracking
  Procedimiento Colorear(vertice, colores_usados)
  Inicio
    // 1. PODA POR COTA SUPERIOR: Si ya igualamos o superamos el mejor C encs
    Si colores_usados >= mejor_C Entonces
      Retornar
    FinSi

    // 2. CASO BASE: Todos los vértices han sido coloreados con éxito
    Si vertice = n + 1 Entonces
      Si colores_usados < mejor_C Entonces
        mejor_C <- colores_usados
      FinSi
      Retornar
    FinSi

    // 3. RAMIFICACIÓN Y EXPLORACIÓN (Aplicando ruptura de simetrías)
    // Intentar asignar colores ya usados
    Para c <- 1 Hasta colores_usados Con Paso 1 Hacer
      Si EsSeguro(vertice, c) Entonces
        color_asignado[vertice] <- c
        Colorear(vertice + 1, colores_usados)
        color_asignado[vertice] <- 0 // deshacer para backtracking
      FinSi
    FinPara

    // Intentar asignar un color nuevo (colores_usados + 1)
    Si EsSeguro(vertice, colores_usados + 1) Entonces
      color_asignado[vertice] <- colores_usados + 1
      Colorear(vertice + 1, colores_usados + 1)
      color_asignado[vertice] <- 0 // deshacer para backtracking
  FinFuncion

```

```
        FinSi
    FinProcedimiento

    // Iniciar el algoritmo con el vértice 1 y 0 colores usados
    Colorear(1, 0)

    Imprimir("El número mínimo de colores necesarios (número cromático) es: ", me
FinProcedimiento
```

Análisis de Coste

- **Complejidad Temporal:** $O(n^n)$ en el peor caso teórico sin podas, reduciéndose drásticamente en la práctica a $O(B^n)$ (donde $B \ll n$) gracias a la ruptura de simetrías por isomorfismo de colores y la acotación por cota superior.
- **Complejidad Espacial:** $O(n)$ debido a la profundidad máxima de la pila recursiva (n niveles de llamadas) y al almacenamiento del vector de colores de tamaño n .

Ejercicio 5: Las Guardias del Sultán de Brunei (Backtracking)

Enunciado

El sultán de Brunei desea colocar N guardias en un palacio de dimensiones $N \times N$ (con N^2 salas cuadradas) de manera que **ningún par de guardias se puedan ver entre sí**.

La visibilidad funciona a través de túneles rectos en la misma **fila, columna o diagonal**. La línea de visión se bloquea únicamente si hay otro guardia en medio. El objetivo es diseñar un algoritmo de **Vuelta Atrás (Backtracking)** para encontrar una colocación válida de los N guardias en las salas.

1. Análisis del Espacio de Búsqueda y Regla de Visión

Para resolver el problema tal y como lo define el sultán, debemos modelar la regla de visión de forma explícita:

- Dos salas (f_1, c_1) y (f_2, c_2) están en la misma línea de visión si comparten fila ($f_1 = f_2$), columna ($c_1 = c_2$) o diagonal ($|f_1 - f_2| = |c_1 - c_2|$).
- Un guardia en la posición A puede ver a un guardia en la posición B si están en la misma línea de visión y **no hay ningún otro guardia en el camino recto entre ellos** que obstruya los túneles.

Estrategia de Vuelta Atrás (Backtracking)

Para colocar exactamente N guardias en un tablero $N \times N$:

- Representamos el tablero como una matriz de dimensiones $N \times N$.
- Intentamos colocar los guardias de forma sistemática. Como debemos colocar N guardias en N filas y no pueden verse directamente, una estrategia eficiente es intentar colocar **exactamente un guardia por fila** (desde la fila 1 hasta la N).
- Para cada celda tentativa $(fila, col)$, lanzamos "rayos" visuales en las direcciones de los túneles para comprobar si hay algún guardia visible sin obstáculos intermedios.

2. Diseño del Algoritmo

Función de Comprobación de Visión (EsSeguro)

Para verificar si es seguro colocar un guardia en la posición (f, c) , exploramos las direcciones de los túneles (norte, sur, este, oeste y las cuatro diagonales).

En cada una de las 8 direcciones, avanzamos paso a paso desde (f, c) hacia el extremo del tablero:

- Si encontramos una celda vacía, seguimos avanzando.

- Si encontramos a otro guardia, significa que se ven directamente (puesto que es el primer obstáculo en esa línea) $\implies$ **No es seguro.**
- Si llegamos al límite del tablero sin encontrar ningún guardia, esa dirección es segura.

Algoritmo de Backtracking Principal

La función recursiva intentará colocar un guardia en la fila actual, probando cada columna de 1 a N . Si es seguro, coloca el guardia temporalmente y avanza a la siguiente fila. Si la exploración falla, deshace la asignación (vuelta atrás).

3. Algoritmo y Pseudocódigo

Función de Factibilidad (Ray-Casting con Bloqueo)

```

Función EsSeguro(tablero, N, fila, col):
    // Definimos las 8 direcciones de los túneles (df, dc)
    Direcciones = [
        (-1, 0), // Norte
        (1, 0),  // Sur
        (0, -1), // Oeste
        (0, 1),  // Este
        (-1, -1), // Noroeste
        (-1, 1), // Noreste
        (1, -1),  // Suroeste
        (1, 1)   // Sureste
    ]

    Para cada (df, dc) en Direcciones:
        paso_f = fila + df
        paso_c = col + dc

        // Avanzamos por el túnel en esa dirección paso a paso
        Mientras (1 <= paso_f <= N y 1 <= paso_c <= N):
            Si (tablero[paso_f][paso_c] == 'GUARDIA'):
                // Encontramos un guardia directamente en la línea de visión
                // (No hay ningún guardia intermedio que lo bloquee ya que es el
                Retornar Falso

            // Si la sala está vacía, el túnel sigue libre. Continuamos explorando
            paso_f = paso_f + df
            paso_c = paso_c + dc

        // Si ningún "rayo" visual ha colisionado con otro guardia, la posición es segura
        Retornar Verdadero

```

Algoritmo de Colocación Recursiva

```

Función ResolverSultan(tablero, N, fila_actual, guardias_colocados):
    // Caso Base: Hemos colocado con éxito las N guardias
    Si guardias_colocados == N:
        Retornar Verdadero

```

```

// Si hemos recorrido todo el tablero y no pusimos las N guardias, fallamos
Si fila_actual > N:
    Retornar Falso

// Intentamos colocar un guardia en la fila_actual probando cada columna
Para col desde 1 hasta N:
    Si EsSeguro(tablero, N, fila_actual, col):
        // 1. Colocar guardia en la sala (fila_actual, col)
        tablero[fila_actual][col] = 'GUARDIA'

        // 2. Intentar colocar el siguiente guardia en la siguiente fila
        Si ResolverSultan(tablero, N, fila_actual + 1, guardias_colocados + 1)
            Retornar Verdadero

        // 3. Backtracking: Deshacer la colocación si no llevó a una solución
        tablero[fila_actual][col] = 'VACÍO'

// Si no se pudo colocar ningún guardia válido en esta fila, retrocedemos
Retornar Falso

```

4. Análisis de Complejidad

- **Temporal:** En el peor de los casos, la complejidad está acotada por $O(N!)$. La función de comprobación de visibilidad tarda $O(N)$ al explorar los túneles rectos de longitud máxima N . Gracias a la poda del backtracking, la mayoría de las ramas inválidas se descartan en las primeras filas, haciendo que el algoritmo sea extremadamente rápido en la práctica.
- **Espacial:** $O(N^2)$ para representar explícitamente el palacio de $N \times N$ con sus salas, más el coste de la pila de llamadas del sistema que consume un espacio de $O(N)$ debido a la profundidad de la recursión.

Ejercicio 6: Ciclos Hamiltonianos con Aristas Prohibidas

Enunciado

Considera un grafo completo no dirigido con N vértices $\{1, \dots, N\}$ y A aristas, donde hay $k < A$ aristas que no se pueden recorrer (prohibidas). Diseña un algoritmo para calcular el número de ciclos hamiltonianos en el grafo que no incluyen las aristas prohibidas.

Un grafo es completo si cada vértice está conectado a todos los demás vértices del grafo. Un ciclo hamiltoniano es un ciclo del grafo que visita cada vértice exactamente una vez. Los ciclos que incluyen las mismas aristas se cuentan solo una vez. Por ejemplo, los ciclos $\{1, 2, 3, 4, 1\}$, $\{1, 4, 3, 2, 1\}$ y $\{2, 3, 4, 1, 2\}$ son todos iguales (se cuentan una vez), mientras el ciclo $\{1, 3, 2, 4, 1\}$ es diferente de los anteriores.

1. Análisis del Problema y Reducción de Simetrías

En un grafo completo no dirigido de N vértices, el número total de ciclos hamiltonianos posibles es $\frac{(N-1)!}{2}$.

Para evitar contar el mismo ciclo físico varias veces en distintas representaciones (rotaciones o inversiones como $\{1, 2, 3, 4, 1\}$ y $\{1, 4, 3, 2, 1\}$), aplicamos dos reglas de **reducción de simetría**:

1. **Fijar el vértice de inicio:** Todo ciclo hamiltoniano válido debe visitar el vértice 1. Por lo tanto, podemos fijar el vértice 1 como el inicio y el final absoluto de todos los caminos explorados. Esto elimina las rotaciones redundantes (ej. $\{2, 3, 4, 1, 2\}$ se reduce a $\{1, 2, 3, 4, 1\}$).
2. **Fijar la dirección (Evitar caminos invertidos):** Para evitar contar un ciclo y su reverso (ej. $\{1, 2, 3, 4, 1\}$ y $\{1, 4, 3, 2, 1\}$), podemos imponer que el primer paso desde el vértice 1 vaya a un vértice vecino u menor que el último paso antes de cerrar el ciclo v (es decir, $u < v$). Sin embargo, en un algoritmo de vuelta atrás es aún más fácil: **simplemente dividimos el resultado final del conteo de caminos dirigidos entre 2**.

2. Diseño del Algoritmo (Vuelta Atrás / Backtracking)

Dado que un grafo completo tiene todas las aristas posibles, representamos las aristas prohibidas mediante una **matriz de adyacencia de prohibición** P de tamaño $N \times N$, donde:

- $P[u][v] = \text{Verdadero}$ si la arista entre u y v está prohibida.
- $P[u][v] = \text{Falso}$ en caso contrario (arista permitida).

Estrategia de Búsqueda:

1. Empezamos en el vértice 1. Marcamos el vértice 1 como visitado.

2. Exploramos recursivamente todos los vértices vecinos no visitados v .
3. Solo transitamos de un nodo actual u a un vecino v si la arista (u, v) **no está prohibida** ($P[u][v] == \text{Falso}$).
4. Cuando hayamos visitado exactamente los N vértices del grafo:
 - Comprobamos si podemos regresar al vértice inicial 1 desde el último nodo de la ruta sin usar una arista prohibida.
 - Si es posible, sumamos 1 a nuestro contador de caminos hamiltonianos válidos orientados.
5. Al finalizar toda la exploración, el número de ciclos hamiltonianos únicos será el contador total dividido por 2 (para eliminar el sentido de giro del ciclo).

3. Algoritmo y Pseudocódigo

Matriz de Prohibición

Para agilizar las consultas, cargamos las k aristas prohibidas en un conjunto rápido o en una matriz booleana:

```

Función CalcularCiclosHamiltonianos(N, Aristas_Prohibidas):
    // 1. Inicializar matriz de prohibición
    P[1..N][1..N] = Falso
    Para cada (u, v) en Aristas_Prohibidas:
        P[u][v] = Verdadero
        P[v][u] = Verdadero

    // Vector para controlar qué vértices ya están en el camino actual
    visitado[1..N] = Falso
    visitado[1] = Verdadero // Fijamos el inicio en el vértice 1

    total_caminos = ContarCaminos(1, 1, N, visitado, P)

    // Cada ciclo físico no dirigido se cuenta dos veces (en sentido horario y an
    Retornar total_caminos / 2
  
```

Función de Conteo Recursiva (ContarCaminos)

```

Función ContarCaminos(nodo_actual, cont_visitados, N, visitado, P):
    // Caso Base: Hemos visitado todos los vértices
    Si cont_visitados == N:
        // Debemos poder cerrar el ciclo volviendo al vértice 1 sin usar una aris
        Si P[nodo_actual][1] == Falso:
            Retornar 1
        Sino:
            Retornar 0

    caminos_validos = 0

    // Probamos todos los posibles siguientes nodos 'siguiente' de 1 a N
  
```

```

Para siguiente de 1 a N:
    Si visitado[siguiente] == Falso y P[nodo_actual][siguiente] == Falso:
        // 1. Decisión tentativa: marcar como visitado
        visitado[siguiente] = Verdadero

        // 2. Exploración recursiva
        caminos_validos = caminos_validos + ContarCaminos(siguiente, cont_vis

        // 3. Backtracking: deshacer decisión para probar otras ramas
        visitado[siguiente] = Falso

Retornar caminos_validos

```

4. Análisis de Complejidad

- **Complejidad Temporal:**

- En el peor de los casos (cuando $k = 0$, es decir, no hay aristas prohibidas), el algoritmo explora todas las permutaciones posibles de los vértices restantes, lo que toma un tiempo de $O((N - 1)!)$.
- Con aristas prohibidas ($k > 0$), el espacio de búsqueda se reduce considerablemente (poda temprana), ya que muchas ramas se descartan inmediatamente cuando se detecta una transición prohibida.

- **Complejidad Espacial:**

- $O(N^2)$ para la matriz booleana de prohibición P .
- $O(N)$ para el vector de visitados y la pila de recursión del sistema (cuya profundidad máxima es exactamente N , que es el número total de vértices).